

Persistent memory architecture for agents

Agent memory is a database problem. Nobody wanted it to be. We measured it anyway.



Harshit Khemani · Kush Ahuja · Madhav Bassi · Kushagra Agrawal
Submitted to Dr. Poonam Sangwan

THE SETUP

An agent that forgets is a tool

An agent that remembers is a colleague. Colleagues need somewhere to keep what they know.

Clear the session

Everything the assistant learned is gone.

Switch the branch

The context does not follow.

Hand over to a teammate

They start from nothing.

THE SUBJECT

Notion built Lore

Open source, MIT, 1.0.0. A shared vault so context survives a cleared session, a new branch, or a handover.

MCP server

Six tools your assistant can call.

CLI

The same services, for people.

Hooks

Loads context at session start. Saves at session end.

We read it at commit `95c3558` . Every claim in this talk comes from that source.

THE INTERESTING PART

It is not a vector store. It is a schema.



Projects

6 properties



Topics

4 properties



Memories

39 properties



Entities

7 properties



Facts

24 properties

Five related tables. A subject-predicate-object fact relation with validity intervals. A canonical entity registry with aliases. This is an ER model, arrived at from the outside.

THE CATCH

The store underneath has no joins

WHAT A NOTION DATABASE GIVES YOU

- Filters and sorts over properties
- Cursor pagination, 100 rows a request
- Relations as arrays of page ids

WHAT IT DOES NOT

- A join
- An aggregate
- A unique index or a check constraint
- A conditional write
- Search that reaches page bodies

Every line here is from the published API reference, not from us.

THE QUESTION

So what does that actually cost?

Not rhetorically. In requests, in bytes, in rows the client has to look at itself, and in states the store will accept that a database would refuse.



METHOD · 1

Build the vault backwards

Labels chosen by looking at what a store returned are worthless. So we generate a **world** first — services, people, owners, incidents, decisions — read the facts off it, then render memories from the facts.

11 885

memories

4 277

facts

1 096

entities

361

questions

The right answer is definitional, not judged. Nobody scored their own homework.

Ten questions Lore itself raises

Read off its own commands and hooks. Each annotated with the minimum relational algebra it needs.

Wake-up context

Everything about an entity

Where did this claim come from

What did we believe on date D

What is true now

Counting and grouping

Find contradictions

Follow a decision chain

Search what a memory says

Two joins deep

45%

need a join

11%

need an aggregate

11%

need recursion

67%

need more than a filter

Three stores, one workload

Notion Data API

An emulator restricted to the documented surface. Every refusal carries the reference page it comes from.

SQLite

The same schema as real tables. Indexes, foreign keys, an FTS5 index over bodies.

PostgreSQL

Range-typed validity, a GiST exclusion constraint, GIN over the text.

We hold no Notion workspace. So we report **counts** — requests, bytes, rows — not wall-clock. A round trip is a round trip whoever carries it.

RESULT ZERO

It gets the right answer

100%

of the workload answered **exactly** — the correct set, nothing added, nothing missing. Same as SQLite. Same as PostgreSQL.

A critique that expected wrong answers would stop here, and it would be wrong to. The Notion API is not a bad store. It is a store that will do the work if you send it enough requests.

RESULT ONE

The question is how many requests

522 686

HTTP requests

vs

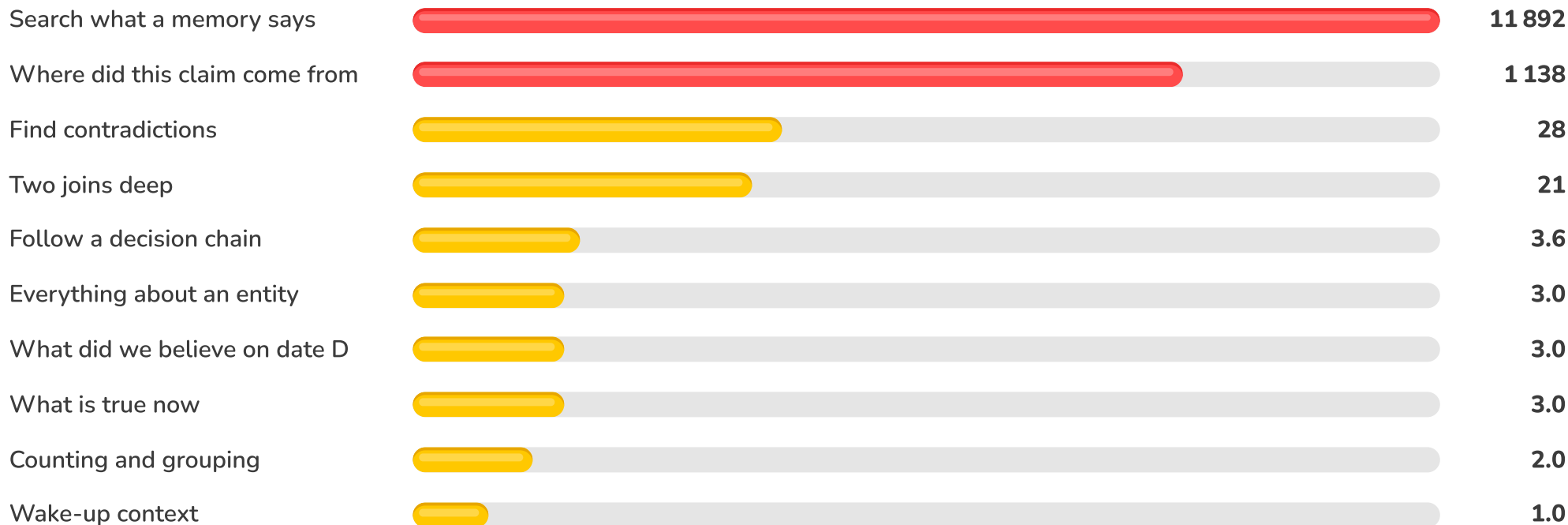
361

SQL statements

Same 361 questions. Same answers. At Notion's documented 3 requests per second, the left column is **48.4 h** of floor before anyone's network is involved.

RESULT ONE

Where the cost lives



Log scale. Two classes cost two or three requests. Two cost more than a thousand.

RESULT TWO

One join

Which claims about this predicate came from a memory written by this author?

The author is on Memories. The fact points at the memory. There is no way to filter Facts by a property of the page its relation points at.

1 138

requests · 6 min floor

vs

1

statement · JOIN

RESULT THREE

Searching what a memory says

A memory's text is page **blocks**. Property filters cannot see blocks, and the search endpoint matches **titles**.

So a complete search over what memories actually say is: list every memory, then fetch every memory's children, then grep locally.

11 891

requests
this vault

66 min

floor
at 3 req/s

110.1 h

floor
at 100× the vault

1

statement
either engine

Lore in practice delegates ranking to Notion's own search, so it is cheaper than this on that one class. It still owns no index — which is why it cannot pre-filter a semantic query, and has no semantic path over Facts at all.

RESULT FOUR

Now the half that is not about speed

A database's central service is not answering questions. It is **refusing to enter states** that would make future answers wrong.



RESULT FOUR

Eight agents, one topic key

Lore upserts by reading the vault, then creating or patching. The API documents no conditional write, so the window between the two cannot be closed.

“Two parallel saves with the same `topicKey` can both find no existing match and both create fresh rows. Notion provides no per-key uniqueness enforcement.”

`src/core/memory-topic-key.ts` – Lore's own source



RESULT FIVE

The constraint that refused to be added

A service has one owner at a time. PostgreSQL can declare that — an exclusion constraint over a range type. Once declared, the write that would create the contradiction is the write that fails.

We tried to add it to the vault as generated. The engine declined to certify data written without it:

```
Key (subject_entity_id, predicate, int8range(..., ..., '[]'::text))=(ent_0003an, owned_by, [177,295)) conflicts with key  
(subject_entity_id, predicate, int8range(..., ..., '[]'::text))=(ent_0003an, owned_by, [205,330))....
```

70

contradictions already in the vault

192

writes PostgreSQL refused

0

writes the vault refused

RESULT SIX

A Boyce-Codd violation that ships

Every Notion database needs a title, and a title cannot be a relation. So Facts carries its subject **twice**: as a string, and as a relation to the canonical entity.

The relation determines the string. Neither is a key. And `mergeEntities` moves relations — it never rewrites titles.

105

facts whose title and relation disagree

2.5% of all facts

40

entities that exist twice

3 867

memories still asserting a closed value

RESULT SEVEN

Two indexes. One text. Two answers.

We asked all three stores for the phrase `timed out`. The corpus says *times out*.

literal substring		0
SQLite FTS5		0
PostgreSQL GIN		669
Notion search		0

PostgreSQL parses the phrase as the single lexeme `'time'`: English stemming turns *timed* into *time*, and *out* is a stopword.

None of them is wrong. A stemmer is a position on what "discusses this" means. The problem is that nothing in any answer says which position was taken.

THE CEILING

Memory systems do not get slow because the prompt grew

TOKENS INJECTED AT SESSION START

775 → 844

Nearly flat. The payload is capped by design.

VAULT IT WAS ASSEMBLED FROM

1 486 → 11 885

Eight times the store. The cost of assembling a bounded prompt is what grows.

A rate limit turns that growth into seconds. Lore mitigates it well — a session debounce, a digest, an off switch — but every mitigation is a cap, and a cap is where a full scan was found to be infeasible.

BEING FAIR

What the trade actually buys

The memory is legible

A teammate opens the vault, reads what the agent believes, and fixes it. In a tool they already use. No system in the literature offers this.

The permissions already exist

A vault inherits Notion's permission model. A bespoke store needs its own, replicated from an identity provider — a famous source of leaks.

There is no infrastructure

No server, no backups, no migration window, no on-call for the memory system itself.

Lore trades every guarantee a database declares for legibility, zero infrastructure and inherited permissions — then re-implements the discarded guarantees as scanners, migrations and locks. Its own source is unusually honest about where the seams are.

THE RECOMMENDATION

The boundary is not where you think

Retrieval is fine: bounded pages, server-side filters, a capped payload. What degrades is **maintenance** — the conflict scan, the entity backfill, the aggregate. Every one of those touches every row.

Keep the vault

It is the system of record because it is legible, and legible is what gets a wrong fact fixed.

Add a read-side projection

A local relational mirror carrying the indexes, the constraints and the joins, rebuilt from the vault.

Humans still edit the vault

Nothing in Lore's design forecloses this. It is an ordinary architecture with an ordinary name.

The field is converging on a schema

Triples with validity intervals. Entity registries with aliases. Provenance edges.
Confidence scores.

That is a database schema. The question every agent-memory system has to answer is not which embedding model to use — it is **which guarantees it is choosing to do without, and what it will do instead.**

Lore answers that more explicitly than most, in comments in its own source.
We put the numbers next to the answer.



Read the paper · `dbms-memory.khe.money`

[Slides](#) · [Paper](#) · [Dataset](#)